Hands-On 3: Dynamic Programming

Competitive Programming and Contests (2025/2026) – Lorenzo Ferrari

1 Overview

This assignment explores dynamic programming techniques in Rust. This report outlines the core algorithmic logic and complexity for Problem 1 (`holiday_planning`) and Problem 2 (`design_a_course`). This report has been generated by Antigravity with Gemini 3.8 Flash and modified by the author.

2 `holiday_planning`: Multiple-Choice Knapsack DP

Given n cities and total vacation days D, spending c days in city i allows visiting the first c days' attractions, yielding profit $\sum_{j=1}^{c} A[i][j]$. We seek the maximum attractions visitable across all cities within at most D days.

- **Algorithmic Logic**:
  - **Grouped Knapsack Formulation**: Each city represents a set of mutually exclusive choices: spending $c \in \{0, 1, ..., D\}$ days with cost c and reward $\text{gain}(i, c) = \sum_{j=1}^{c} A[i][j]$. This maps directly to the **Multiple-Choice Knapsack Problem** with capacity D.
  - **State & Transitions**: Let $\text{dp}[k]$ be the maximum attractions visitable within k days ($0 \leq k \leq D$). For each city i, we accumulate prefix attractions $\text{acc} = \sum_{t=1}^{c} A[i][t]$ on-the-fly for each choice $c \in [1, D]$. To ensure at most one choice of days is selected per city, transitions pull from the previous city's DP state prv:

$$\text{dp}[k] \leftarrow \max(\text{dp}[k], \text{prv}[k - c] + \text{acc}) \quad \forall k \in [c, D]$$

  - **Space Optimization**: By maintaining only the current table dp and the previous city's table prv, auxiliary space is reduced to $\mathcal{O}(D)$.
- **Complexity**:
  - **Time Complexity**: For each of the n cities, c runs from 1 to D, and the inner loop updates budgets $k \in [c, D]$, executing $\sum_{c=1}^{D}(D - c + 1) = \mathcal{O}(D^2)$ steps per city. Total time is $\mathcal{O}(nD^2)$.
  - **Space Complexity**: $\mathcal{O}(D)$ auxiliary space for the rolling buffers, beyond $\mathcal{O}(nD)$ input storage.

3 `design_a_course`: 2D Longest Increasing Subsequence via Fenwick Tree

Given n topics, each with beauty b_i and difficulty d_i, we want to select the maximum number of topics that can be arranged in strictly increasing order of both beauty and difficulty: $b_{i_1} < ... < b_{i_k}$ and $d_{i_1} < ... < d_{i_k}$.

- **Algorithmic Logic**:
  - **2D Dominance / LIS**: Because the course curriculum can order the chosen topics arbitrarily, this corresponds to finding the longest chain under strict 2D coordinate dominance ($(b, d) < (b', d') \Leftrightarrow b < b' \wedge d < d'$).
  - **Sorting & Tie-Breaking**: We sort topics primarily by beauty b ascending and secondarily by difficulty d **descending**:

$$(b_1, d_1) < (b_2, d_2) \Leftrightarrow (b_1 < b_2) \vee (b_1 = b_2 \wedge d_1 > d_2)$$

    Ordering ties in reverse on d ensures that points with identical b appear in decreasing order of difficulty. Hence, an increasing sequence on d cannot select multiple topics with the same b, naturally enforcing strict inequality $b_{\text{prev}} < b_{\text{cur}}$.
  - **Coordinate Compression & Fenwick Tree**: Difficulties d_i are deduplicated and mapped to ranks $[1, U]$ ($U \leq n$). A Fenwick Tree maintains prefix maximums of DP values over y-ranks. For each sorted point with rank y:

$$\text{dp} = 1 + \text{FT.query}(y - 1), \quad \text{FT.update}(y, \text{dp})$$

    The maximum DP value found across all points yields the optimal chain length.
  - **(Note on Fixed Order)**: If the input order was fixed (3D dominance: index, beauty, difficulty), CDQ divide-and-conquer in $\mathcal{O}(n \log^2 n)$ would be required (implemented in `design_a_course_if_order_is_fixed.rs`). Since reordering is permitted, the 2D sweep suffices.
- **Complexity**:
  - **Time Complexity**: Sorting points and coordinate compression take $\mathcal{O}(n \log n)$ time. Sweeping through n points with Fenwick queries and updates takes $\mathcal{O}(n \log n)$. Total time is $\mathcal{O}(n \log n)$.
  - **Space Complexity**: $\mathcal{O}(n)$ auxiliary space to store points, compressed coordinates, and the Fenwick tree.